

Applied Human Computer Interaction (AHCI)

Asif Fazwani

Chapter 7 – Design Rules

Introduction to Design Rules

Core challenge: Predict **usability impact** of design decisions

Solution: Use **design rules**

Help designers create **user-friendly systems early**

Example: If you design a **mobile app like TikTok**, design rules help decide:

- Where to place buttons
- How simple navigation should be

What Are Design Rules?

Rules designers follow to improve usability

Act as **guidance for better UX decisions**

Reduce chances of building confusing interfaces

Example: “Keep navigation simple” → why apps like Instagram use bottom navigation bars

Two Key Dimensions of Design Rules

Authority Must follow (strict) vs optional (suggested)

Generality Broad (applies everywhere) vs specific (limited use)

Think of it like:

- “Must submit assignment before deadline” (high authority)
- “Study at night for better focus” (low authority)

Types of Design Rules

Principles: General + flexible

Standards: Strict + specific

Guidelines: Recommended + adaptable

Analogy:

- Principles = Life advice
- Guidelines = Study tips
- Standards = Exam rules



Types of Design Rules

Principles (Abstract & Flexible) High generality, low authority

- Based on **human behavior & psychology**
- Work across different technologies

Example: “Reduce cognitive load” → Apps like Google Search keep UI minimal

Netflix automatically plays the next episode

No need to decide “what to do next”

Reduces thinking → keeps users engaged effortlessly

Types of Design Rules

Standards (Strict Rules)

- High authority (must follow)
- Specific to context/platform
- Often enforced by organizations

Example: **App Store guidelines** by Apple Inc. If violated → app gets rejected

Google Play Store Policies

Payment Security (PCI Standards)

Types of Design Rules

Guidelines (Flexible Suggestions)

- Medium generality
- Not mandatory but helpful
- Often based on best practices

Example: “Use readable fonts” → Apps like Spotify use clean typography

Conflicts Between Design Rules

- Not all rules can be followed together
- Designers must make **trade-offs**

Example: More features vs simple UI → Snapchat sometimes criticized for complexity

When to Use Design Rules?

- Most useful in **early design stages**
 - Requirements
 - Architecture

- Help reduce **bad design choices early**

Example: Designing a startup app → better to fix UX early than after launch

- Rules may be **platform-specific**
- Applying them blindly → poor design

Example: Desktop-style UI doesn't work well on mobile apps

Principles to Support Usability

- Most **abstract design rules**
- Help improve overall **usability of systems**
- Derived from:
 - Past successful designs
 - Human behavior understanding

Example: Why apps like TikTok feel easy → built on strong usability principles

Why Principles Matter

- Explain **why designs work (or fail)**
- Provide **repeatable design guidance**
- Can be applied across different systems

The simplicity of Google Search works across devices because of universal principles

Principles to Support Usability

Three Key Usability Principles

- **Learnability:** the ease with which new users can begin effective interaction and achieve maximal performance.
- **Flexibility:** the multiplicity of ways the user and system exchange information
- **Robustness:** the level of support provided the user in determining successful achievement and assessment of goal-directed behavior

Principles to Support Usability

Learnability (Easy to Learn)

- How quickly a new user can start using a system
- Focus: **first-time user experience**

Example: *Instagram is easy to start:* Scroll feed instantly, No training needed

Flexibility (Multiple Ways to Interact)

- Different ways to perform the same action
- Supports user preferences

Example: *WhatsApp allows:* Text, Voice notes, Video calls

Robustness (Support & Feedback)

- Helps users: Achieve goals, Know if actions succeeded

Example: Google Maps: Shows route progress, Alerts if you go wrong

Learnability Sub-Principles

1. **Predictability:** determining effect of future actions based on past interaction history
2. **Synthesizability:** assessing the effect of past actions immediate vs eventual honesty
3. **Familiarity:** how prior knowledge applies to new system guess ability and affordance
4. **Generalizability:** extending specific interaction knowledge to new situations
5. **Consistency:** likeness in input/output behavior arising from similar situations or task objectives

These make systems **easy, intuitive, and fast to learn**

Principles to Learnability - Predictability

Predictability (Future Actions)

- Users can **predict what will happen next**
- Based on: Past interaction, Visible interface

Example: On YouTube, Click video → it plays

Operation Visibility (Part of Predictability)

- Users should see **what actions are possible**
- Prefer **recognition over memory**

Example: On Spotify Play, pause, skip buttons are always visible

Users don't need to remember controls

Principles to Learnability - Synthesizability

Synthesizability (Past Actions)

- Users understand **what just happened**
- Can evaluate results of actions

Example: On Instagram You post → instantly see likes/comments, Immediate feedback builds understanding

Immediate vs Eventual Feedback

- **Immediate honesty** → instant feedback
- **Eventual honesty** → delayed feedback

Example: Drag file on desktop → see it move instantly vs command-based systems → need to check manually

Immediate feedback is better for learning

Principles to Learnability - Familiarity

Familiarity: Use **real-world knowledge & past experience** and makes system feel intuitive

Example: Cart icon in Amazon, Feels like real shopping

Social media icons are universal  Like → Instagram  Retweet → Twitter

Users don't need explanation

Affordances (Visual Clues) Objects suggest how to use them

Example:

- Button → looks clickable
- Slider → looks draggable

Seen in apps like TikTok filters

Principles to Learnability - Generalizability

Generalizability: Users apply knowledge to **new situations**

Example: Swipe gestures: TikTok, Instagram

Learn once, use everywhere

Cross-App Generalizability Same features across apps

Example: Copy-Paste works in:

- Microsoft Word
- Google Docs

Reduces learning time

Principles to Learnability - Consistency

Consistency: Similar actions → similar results - Most important UX principle

Example: ❤️ button behaves same across: Instagram, Facebook

Consistency – Real Insight

- Not always simple
- Must define “consistent with what?”

Types:

- Within app
- Across apps
- Real-world consistency

When Consistency Goes Wrong Too much consistency can be bad

Example: Old alphabetical keyboards → inefficient → replaced by QWERTY

Best design ≠ always most “logical”

Flexibility Sub-Principles

1. **Dialogue initiative:** freedom from system-imposed constraints on input dialogue system vs. user pre-emptiveness
2. **Multithreading:** ability of system to support user interaction for more than one task at a time concurrent vs. interleaving; multimodality
3. **Task Migratability:** passing responsibility for task execution between user and system
4. **Substitutivity:** allowing equivalent values of input and output to be substituted for each other representation multiplicity; equal opportunity
5. **Customizability:** modifiability of the user interface by user (adaptability) or system (adaptivity)

Principles to Flexibility - Dialogue initiative

Dialogue initiative: Who controls interaction?

System-driven (pre-emptive) or User-driven

Good design → more user control

Example: Annoying popups blocking actions (system-driven) vs free browsing in YouTube

When System Control is Needed

Sometimes system must restrict user:

- Safety
- Collaboration

Example: Google Docs prevents editing conflicts in Google Docs

Principles to Flexibility - Multi-Threading

Multi-Threading: Users can handle **multiple tasks at once**

Types: Concurrent (simultaneous) and Interleaved (switching tasks)

Example: Chat + watch videos + browse → Common user behavior

Apps like: TikTok, WhatsApp

Multi-Modality

- Multiple ways to interact:
 - Touch
 - Voice
 - Keyboard

Example: Google Assistant Speak OR type commands

Principles to Flexibility - Task Migratability

Task Migratability: Tasks can shift between: **User ↔ System**

Example: Google Maps suggests:

- Best routes
- Traffic updates
- User can:
 - Accept suggestion
 - Choose alternative route

System guides → user decides

Human + AI collaboration

Principles to Flexibility - Substitutivity

Substitutivity: Same task, different methods

Example: In calculators or spreadsheets: Microsoft Excel

Enter value directly OR use formula

Reduces mental effort

Representation Multiplicity Same data shown in different formats

Example: In Google Sheets Numbers → table OR graph Helps different thinking styles

Equal Opportunity: If you can see it, you can use it

Example: Drag & drop files in Windows File Explorer Output becomes input.

Principles to Flexibility - Customizability

Customizability: Interface can be modified

Types: **Adaptability** (user changes) **Adaptivity** (system auto-adjusts)

Adaptability (User Control) User customizes UI

Example: Themes in Spotify or chat wallpapers in WhatsApp

Adaptivity (System Smartness) System learns from user behavior

Example: Recommendations in Netflix AI-driven UX

Robustness Sub-Principles

1. **Observability:** ability of user to evaluate the internal state of the system from its perceivable representation browsability; defaults; reachability; persistence; operation visibility.
2. **Recoverability:** ability of user to take corrective action once an error has been recognized reachability; forward/backward recovery; commensurate effort
3. **Responsiveness:** how the user perceives the rate of communication with the system; Stability
4. **Task conformance:** degree to which system services support all of the user's tasks, task completeness; task adequacy

Principles to Robustness - Observability

Observability: Users can see system state

Example: Typing indicator in WhatsApp “User is typing...” → system transparency

Browsability Users can explore system Scrolling feeds in Instagram

Defaults Pre-filled values reduce effort. Auto-filled forms in Google Chrome

Reachability Can users navigate anywhere? Back button in almost every app, Prevents getting “stuck”

Persistence Information remains visible Unread message badge in Gmail

Principles to Robustness - Recoverability

Recoverability Users can fix mistakes

Types: Undo (backward), Fix forward

Example: Undo delete in Microsoft Word

Commensurate Effort Hard to undo → should be hard to do

Example: Deleting account requires multiple steps, Prevents accidental loss

Principles to Robustness – Responsiveness & Task Conformance

Responsiveness System should respond quickly

Example: Instant loading in TikTok Slow apps = users leave

Task Conformance System supports real user tasks

Types: Task completeness and Task adequacy

Example: Uber → Matches real-world goal: “Get a ride quickly”

Activity 7

<https://tinyurl.com/Spring2026-Activity7>

Question 1

Which sub-principles of learnability are violated? (choose & justify):

Predictability Synthesizability Familiarity Generalizability
Consistency

- **Confusing icons**
 - Familiarity (icons not recognizable)
 - Consistency (no standard patterns)
- **No confirmation after submission**
 - Synthesizability (user can't see result of action)
 - Predictability (unsure what happened)

Question 2

Which sub-principles of Flexibility are violated? (choose & justify):

Dialog initiative Multi-threading

Task migratability Substitutivity Customizability

Single upload option

Substitutivity (no alternative methods)

Blocking popups

Dialog Initiative (system controls interaction)

Question 3

Which sub-principles of Robustness are violated? (choose & justify):

Observability Recoverability Responsiveness Task conformance

No undo → Recoverability (cannot fix mistakes)

Slow system/no feedback

Responsiveness

Observability (no system status visible)

Question 4

Redesign one problematic feature of FLEXPlus+.

Explain your design Mention which principles & sub-principles are improved

Response:

Add upload progress bar + confirmation message

Improves:

- Synthesizability (see result)
- Observability (system status visible)
- Responsiveness (feedback during delay)

Standards in HCI

Standards = official design rules

Set by: National bodies International organizations to ensure consistency & quality across systems

International Organization for Standardization (ISO)

British Standards Institution (BSI)

Standards in HCI

Hardware Standards

- Based on:
 - Ergonomics
 - Human physiology
- Stable and well-defined

Software Standards

- Based on:
 - Psychology
 - Cognitive science
- More abstract and evolving

Key Difference

- Hardware → **Precise rules**
- Software → **General guidelines**

Example:

- Keyboard layout → fixed standard
- UI design → more flexible

Hardware: Expensive to change , Standards are stable

Software: Easy to update, Standards evolve quickly

Example: App UI updates in Instagram happen frequently

Guidelines in HCI

- Guidelines = **general design recommendations**
- Less strict than standards
- Help designers make **better UI decisions**
- Most commonly used in **UI/UX design**

Example: “Use clear icons and labels” in apps like Instagram

- **Standards** → strict, must follow
- **Guidelines** → flexible, suggested

Guidelines allow **creativity + adaptation**

Guidelines in HCI

- **Abstract guidelines (principles)** applicable during early life cycle activities
High-level, conceptual
Focus on **user experience goals**
- **Detailed guidelines (style guides)** applicable during later life cycle activities
Low-level, implementation-focused
Define **exact UI behavior/layout**

Guidelines in HCI

Apple Guidelines Example

By Apple Inc.

Abstract Principle: “Be consistent”

Detailed Rule: “Noun → Verb order”

- Select object → then action

Example: Select file → click delete

Microsoft Guidelines Example

Abstract Principle:

Provide clear feedback to users

Users should always know **what is happening**

Concrete Rule:

Show **visual feedback for every action**

Loading indicators

Progress bars

Status messages

Golden Rules & Heuristics

- Simple, practical design rules
- Easy to remember checklist
- Help in: Design and Evaluation
- Useful check list for good design
- Better design using these than using nothing!

Shneiderman's Golden Rules

Rule 1 – Strive for consistency Same actions → same results

Icons & navigation in Instagram stay consistent

Rule 2 – Enable frequent users to use shortcuts Speed up frequent tasks

Keyboard shortcuts in Google Docs (Ctrl + C, Ctrl + V)

Rule 3 – Offer informative feedback Every action → system response

Typing indicator (“...”) in WhatsApp

Rule 4 – Design dialogs to yield closure Clear task completion

“Order placed successfully” in Amazon

Shneiderman's Golden Rules

Rule 5 – Offer error prevention and simple error handling

Prevent mistakes before they happen - Password rules in Gmail

Rule 6 – Permit easy reversal of actions

Allow reversal of actions - Undo send” in Gmail

Rule 7 – Support internal locus of control User feels in control

Skip/seek controls in YouTube

Rule 8 – Reduce short-term memory load Don't force users to remember - Auto-suggestions in Google Search

Norman's 7 Principles

- 1. Use both knowledge in the world and knowledge in the head**
Don't rely only on memory, also provide **visible cues + learned skills**
Example: Google Maps Shows routes (knowledge in world) and User knows how to read maps (knowledge in head)
Reduces need to memorize directions
- 2. Break complex tasks into simple steps and Reduce thinking effort**
Example: Uber just: Enter location Confirm ride
No need to call drivers manually
- 3. Make Things Visible: Bridge the gulfs of Execution and Evaluation.**
Show what actions are possible
Show system status clearly
Example: Spotify Play, pause, skip buttons always visible
User instantly knows what to do

Norman's 7 Principles

- 4. Get the Mapping Right** Controls should match expected results
Example: Instagram Swipe left/right → moves between posts
Natural, intuitive behavior
- 5. Use Constraints** Prevent incorrect actions
Example: Gmail - “Send” disabled if no recipient
Stops user from making mistakes
- 6. Design for Error** Assume users will make mistakes, provide recovery options
Example: Google Photos
Deleted photos go to **Trash** Easy recovery
- 7. Standardize** Use familiar patterns across systems
Example: YouTube - Same play/pause icons used everywhere
Users don't need to relearn

HCI Design Patterns

- Reusable design solutions
- Based on **successful past designs**
- Help solve **common UI problems**
- Don't reinvent, reuse what works

Solution to a recurring problem in a specific context

It Includes: Problem, Solution, Context, Rationale

HCI Pattern Example

Light on two sides of room Problem: Dark rooms feel unused
Solution: Windows on two sides Same idea → applied in UI

UI Problem: Users feel: Lost, Confused Disoriented

UI Solution: Provide: Navigation cues, Clear structure, Visible options

Real UI Example In Amazon: Breadcrumbs → *Home > Electronics > Laptops* (Back button, Home icon)

This is like adding “windows” in UI → Users always know **where they are**

Why This Pattern Matters

- Users don't browse linearly they jump, explore, get lost
- Safe navigation = confidence
- Patterns are connected
- One pattern leads to another
- Forms a **design system / ecosystem**

Example: **Navigation → Breadcrumbs → Back button**

YouTube:

Back button

Home feed

History

Easy recovery from “lost scroll”

Summary

Principles for usability

- repeatable design for usability relies on maximizing benefit of one good design by abstracting out the general properties which can direct purposeful design
- The success of designing for usability requires both creative insight (new paradigms) and purposeful principled practice

Using design rules

- standards and guidelines to direct design activity